



Operating system and system programming (25CS2104E)

Multithreaded Linux Application Using POSIX Threads and Mutexes

Review-I

Under the Esteemed Guidance of

Raghupathi M
CSE Department

Team Members: 23

2520090110 - Aditya Vallala

2520080067 - Dheeraj Naidu

2520080054 - Surapaneni karthikeya

- Abstract
- Introduction
- Hardware and Software requirements
- File-System Abstractions
- File-I/O System Calls and Semantics
- Structured Data and Robust I/O
- Threads, Mutex and File I/O
- System Architecture
- Demo and Output Evidence
- Coverage Summary
- Conclusion

Abstract

The Multithreaded Linux Application Using POSIX Threads and Mutexes is designed to demonstrate the concepts of concurrent programming in a Linux environment. The application uses POSIX Threads (Pthreads) to execute multiple tasks concurrently, improving responsiveness and resource utilization. Since multiple threads may access shared resources simultaneously, mutexes (mutual exclusion locks) are used to prevent race conditions and maintain data consistency. The project demonstrates thread creation, execution, synchronization, shared-resource management, and proper thread termination. It provides a practical understanding of how multithreading and synchronization mechanisms are implemented in Linux using the C programming language.

The application uses a shared file as the common resource. Threads write and read fixed-size structured records using Linux file-I/O system calls (open, read, write, lseek, pread, close), while a mutex protects the shared file offset. The project also analyzes the file-system abstractions behind these calls: inodes, directory entries, the virtual file system (VFS) layer and mount points.

Introduction

Multithreading is an important concept in modern operating systems that allows a single process to perform multiple tasks concurrently. A thread is a lightweight unit of execution within a process, and multiple threads can share the same memory and resources. This makes multithreading useful for applications that need better performance, responsiveness, and efficient CPU utilization. Linux provides strong support for multithreaded programming through the POSIX Threads (Pthreads) library. Functions such as `pthread_create()`, `pthread_join()`, and `pthread_exit()` are used to create, manage, and terminate threads. When multiple threads access a common resource, problems such as race conditions can occur. To avoid these problems, the project uses mutexes. A mutex allows only one thread at a time to access a critical section of code. This ensures that shared data is modified safely and consistently. The project therefore focuses on implementing a Linux application that demonstrates thread creation, concurrent execution, synchronization using mutexes, critical-section protection, and safe resource management.

In Linux, files are accessed through file descriptors, and the threads of a process share the descriptor table and the file offset. The project therefore lets threads write structured records to one shared file, protected by a mutex, using file-I/O system calls (`open`, `read`, `write`, `lseek`) that rely on inodes, directory entries, the VFS layer and mount points.

Hardware and Software requirements

Hardware requirements :

Component	Requirement
Processor	Intel/AMD Dual-Core or above
RAM	Minimum 4 GB
Storage	Minimum 10 GB free space
Keyboard	Standard keyboard
Display	Standard monitor
Network	Optional

Software requirements :

Software	Requirement
Operating System	Linux (Ubuntu recommended)
Programming Language	C
Compiler	GCC
Thread Library	POSIX Threads (Pthreads)
Development Tools	GCC, Make, Text Editor/IDE
Terminal	Linux Terminal
Libraries	pthread.h, standard C libraries

Additional libraries: fcntl.h, unistd.h, sys/stat.h, errno.h, stdint.h | File-system inspection tools: strace, stat, ls -i, findmnt, hexdump

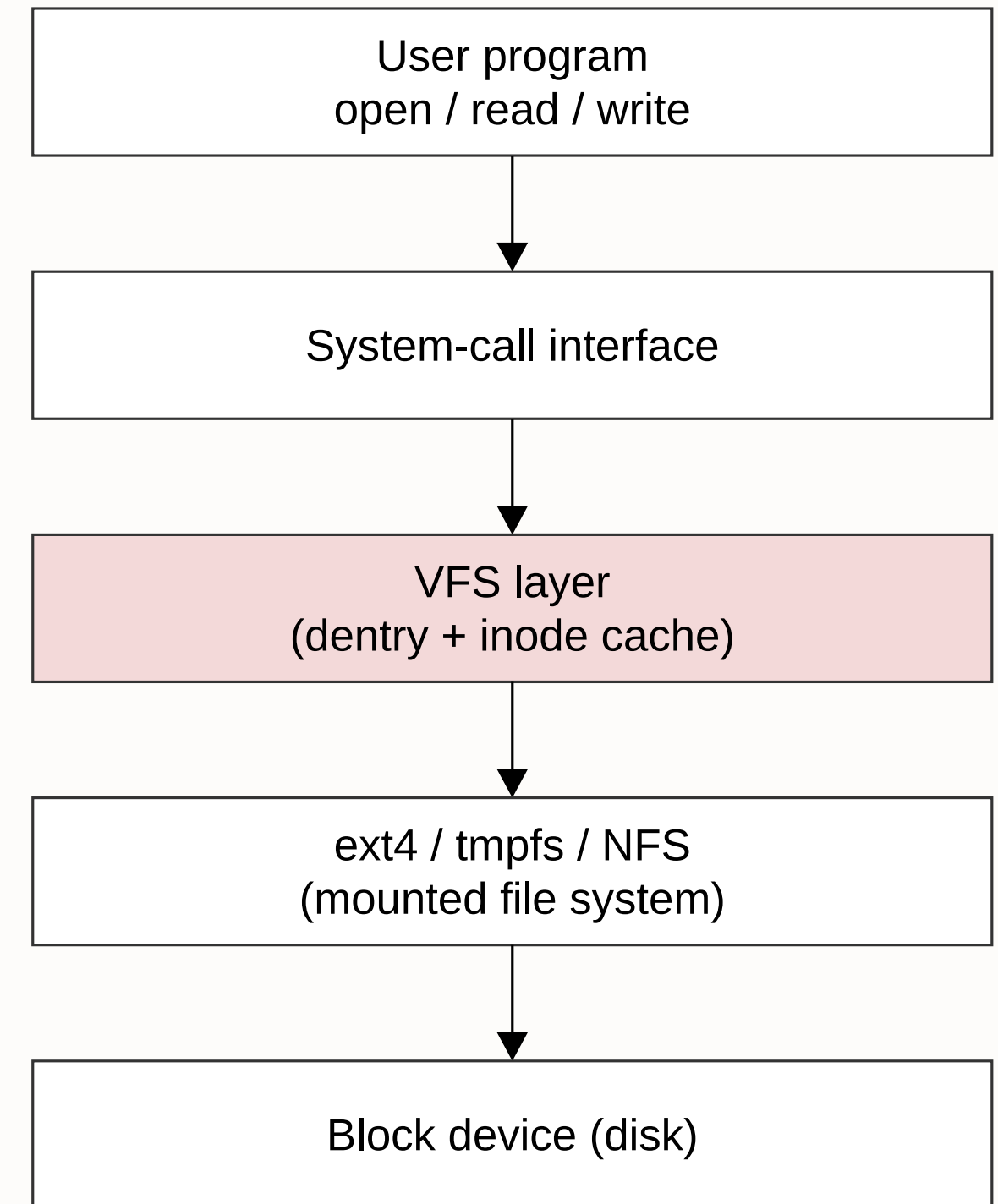
File-System Abstractions

Inode: kernel object that stores a file's metadata: size, owner, permissions, timestamps and block pointers. It has a number but no name. Shown by `ls -li`, `stat`, or `st_ino` from `fstat()`.

Directory entry (dentry): maps a file name to an inode number. A directory is a special file made of such entries, and several names (hard links) can point to one inode.

Virtual File System (VFS): common layer that makes open, read and write behave the same on ext4, tmpfs, NFS and others, using superblock, inode, dentry and file objects.

Mount point: directory where a file system is attached to the single directory tree. Path lookup crosses mount points transparently (`findmnt`, `df`).



File-I/O System Calls and Semantics

Kernel structures: per-process **fd table** points to an **open file description** (offset and flags), which points to the **inode**. Threads share the fd table, so they share one file offset. This is why the mutex is needed.

`open(path, flags, mode):` O_RDONLY | O_CREAT | O_TRUNC | O_APPEND. Returns an fd, or -1 with errno set.

`read / write:` may transfer fewer bytes than requested (short count); read returns 0 at end of file; -1 with EINTR means retry.

`lseek(fd, off, whence):` SEEK_SET, SEEK_CUR, SEEK_END. Moves the offset without any I/O and enables random access.

`pread / pwrite:` read or write at an explicit offset without changing the shared offset.

`fstat` queries the inode, `fsync` flushes data to disk, `close` releases the descriptor.

Structured Data and Robust I/O

```
typedef struct {  
    uint32_t magic;    /* 0xCAFEBAFE */  
    uint32_t thread_id;  
    uint64_t seq;  
    char      msg[48];  
} record_t; /* 64 bytes */
```

Fixed-size records: record N lives at offset $N \times \text{sizeof}(\text{record_t})$, so `pread()` gives random access. Fixed-width types keep the layout predictable.

Robustness checklist: handle short reads/writes and `EINTR`; check every return value (`perror/strerror`); treat `read() == 0` as end of file; validate magic on read; always `close(fd)`, also on error paths.

```
ssize_t write_all(int fd,  
                  const void *buf, size_t n)  
{  
    const char *p = buf; size_t left = n;  
    while (left > 0) {  
        ssize_t w = write(fd, p, left);  
        if (w < 0) {  
            if (errno == EINTR) continue;  
            return -1;  
        }  
        p += w; left -= w;  
    }  
    return n;  
}
```

Threads, Mutex and File I/O

```
pthread_mutex_lock(&file_lock);  
rec.seq = ++global_seq;  
lseek(fd, 0, SEEK_END);  
write_all(fd, &rec, sizeof rec);  
pthread_mutex_unlock(&file_lock);
```

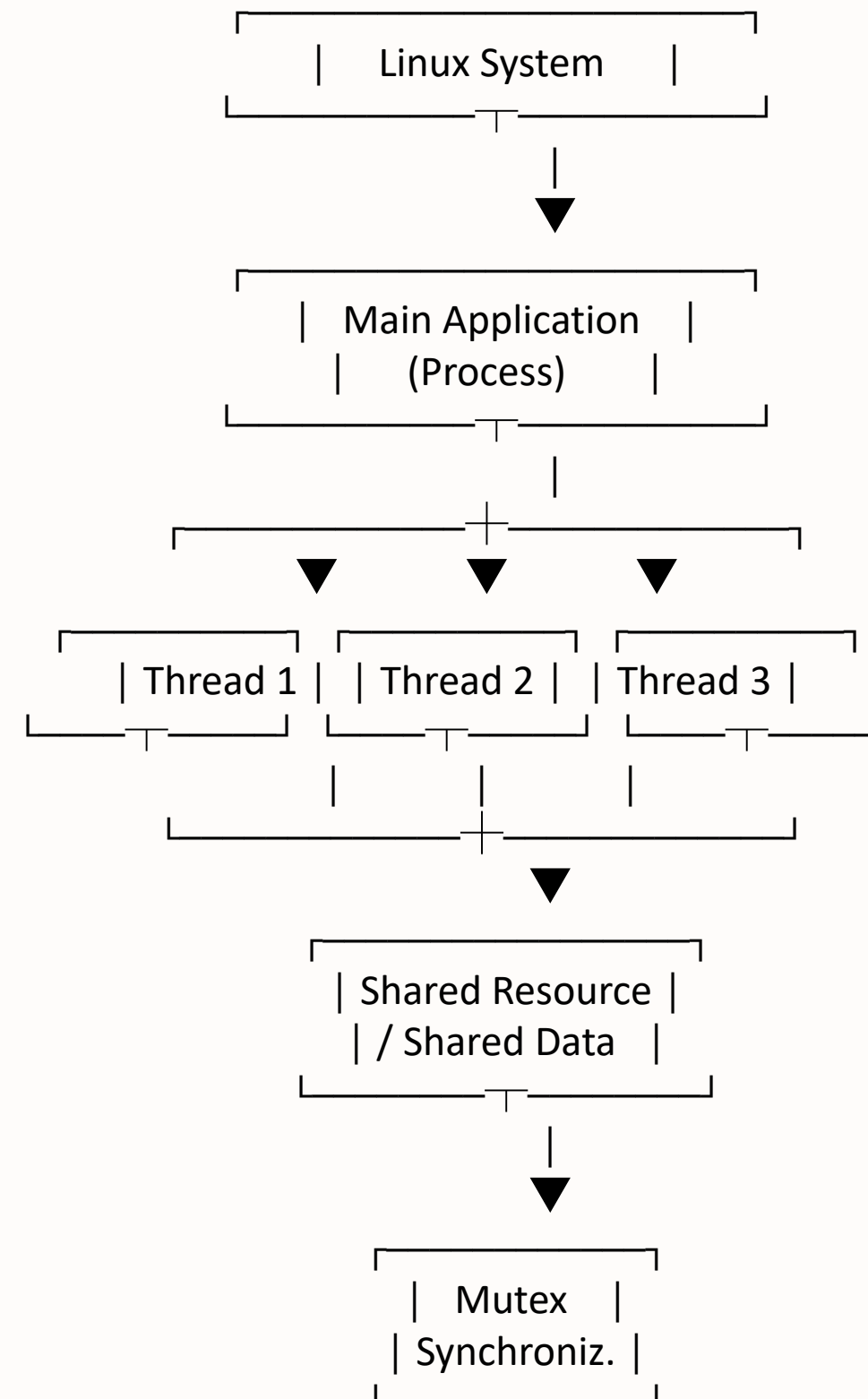
Without the mutex: the shared offset causes interleaved or overwritten records (a demonstrable race).

With the mutex: every record is whole and sequence numbers are unique.

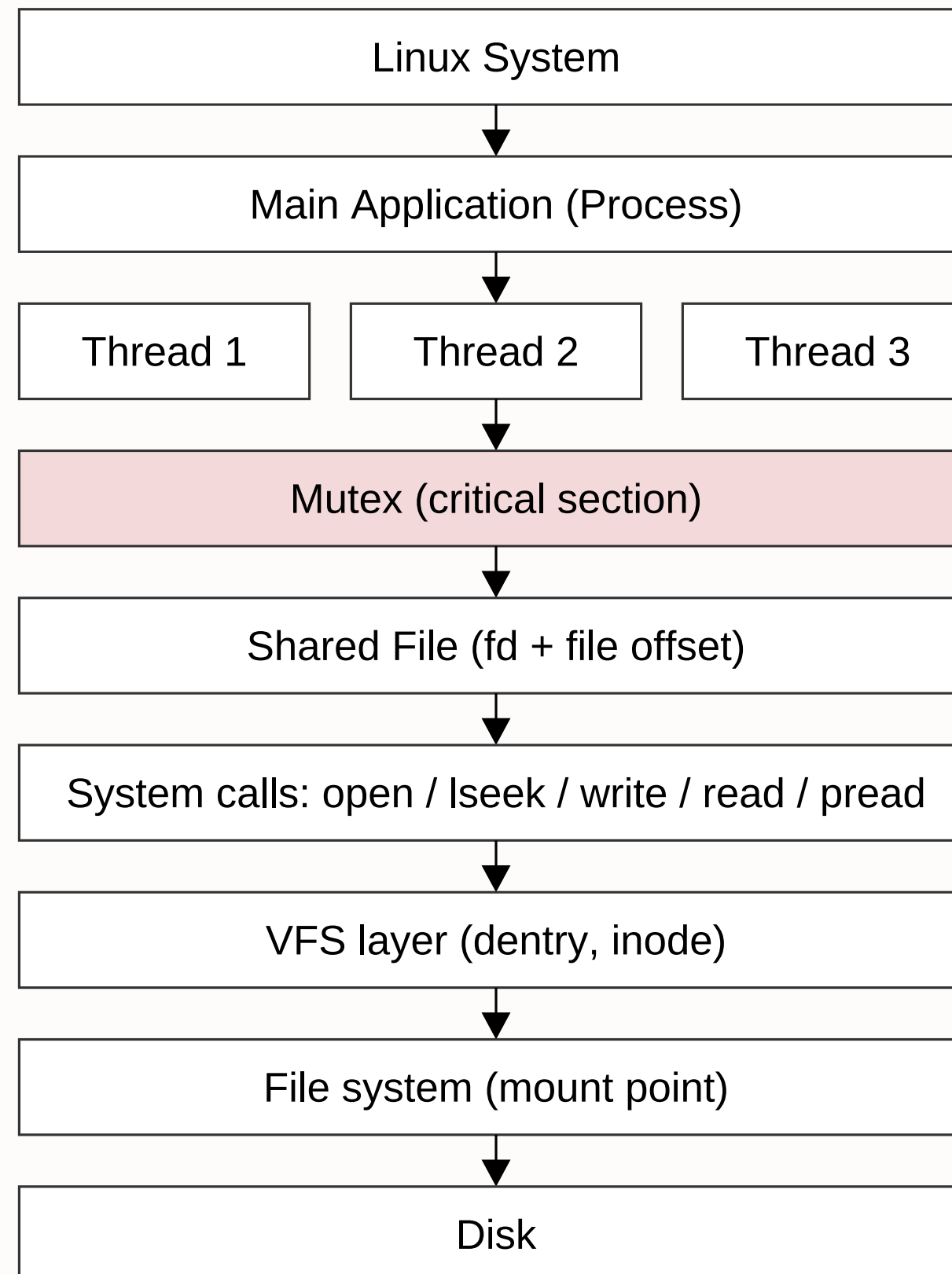
Program flow

1. main() opens data.bin (O_RDWR | O_CREAT | O_TRUNC).
2. Creates 3 threads; each appends N records under the mutex.
3. pthread_join() waits for all threads.
4. Verification: reopen the file; fstat() size must equal 3N x sizeof(record_t); pread() every record and check magic and unique seq.
5. close(fd) and pthread_mutex_destroy().

System Architecture



System Architecture with File Layer



Demo and Output Evidence

Command	What it shows
<code>strace -f ./app</code>	Real openat, lseek, write and read syscalls issued by the threads
<code>stat data.bin</code>	Inode number, size, blocks, permissions, timestamps
<code>ls -li data.bin</code>	Inode number linked to the directory entry (name)
<code>ln data.bin link.bin; ls -li</code>	Two names (dentries) share one inode (hard link)
<code>findmnt -T data.bin</code>	Mount point and file system type that holds the file
<code>hexdump -C data.bin</code>	Structured records: magic, thread_id, seq, msg
Run with and without mutex	Corrupted file versus consistent file (race demonstration)

Coverage Summary

Topic	Where it is covered
Inodes and directory entries	File-System Abstractions; stat, ls -li and hard-link demo
Virtual file system layer	File-System Abstractions; architecture with file layer
Mount points	File-System Abstractions; findmnt demo
Read / write / seek semantics	File-I/O System Calls: fd table, shared offset, short counts, lseek, pread
File-I/O system calls	System Calls and Threads, Mutex and File I/O slides; strace demo
Structured data, robustly	Structured Data and Robust I/O: record_t, write_all, EINTR, validation

Conclusion

The Multithreaded Linux Application Using POSIX Threads and Mutexes successfully demonstrates the fundamental concepts of multithreaded programming in Linux. The project shows how multiple threads can execute concurrently and how POSIX Threads can be used to create and manage them. The use of mutexes provides safe access to shared resources and prevents race conditions and data inconsistency. Overall, the project provides practical knowledge of thread creation, concurrent execution, synchronization, critical sections, mutex locking and unlocking, and thread termination. These concepts are essential for developing efficient and reliable Linux applications and form an important foundation for advanced operating-system and concurrent-programming applications.

The project also demonstrates the file-system side of the operating system: inodes, directory entries, the VFS layer and mount points, the semantics of file descriptors and of read, write and seek, and robust reading and writing of structured data with proper error handling.

Thank you